
NOTE

In the official documentation of pygame, every image, including the application window, is known as a surface. I'll use more specific terms: I will refer to the application window simply as a window and to any picture loaded from an external file as an image. I reserve the term surface for any picture drawn on the fly.

We then tell the program to draw the ball ❷ every time we go through the main loop. We specify the location representing the position to place the upper-left corner of the image's bounding rectangle, typically as a tuple of x- and y-coordinates.

The function name `blit()` is a very old reference to the words *bit block transfer*, but in this context it really just means “draw.” Since the program loaded the ball image earlier, pygame knows how big the image is, so we just need to tell it where to draw the ball. In [Listing 5-2](#), we give an x value of 100 and a y value of 200.

When you run the program, on each iteration through the loop (30 times per second) every pixel in the window is set to black, then the ball is drawn over the background. From the user's point of view, it looks like nothing is happening—the ball just stays in one spot with the upper-left corner of its bounding rectangle at location (100, 200).

Detecting a Mouse Click

Next, we'll allow our program to detect and react to a mouse click. The user will be able to click on the ball to make it appear somewhere else in the window. When the program detects a mouse click on the ball, it randomly picks new coordinates and draws the ball at that new location. Instead of using hardcoded coordinates of (100, 200), we'll create two variables, `ballx` and `bally`, and refer to the coordinates of the ball in the window as the tuple `(ballx, bally)`. [Listing 5-3](#) provides the code.